RIGHTSHIFT · INTUITIOLABS

# Project Playbook

How a client project runs from the first sales call to release: who does what, which skills they use, where every artifact is stored, and the checks that must pass before code ships.

Version 1 · 14 October 2026 · Worked example: Compass (client contact: Steve)
Applies to the `rs-*` plugins from the `rightshift` marketplace, v4 and the upcoming v5 (multi-repo).

# Contents


1. The model in one page
2. Roles and responsibilities
3. Where everything is stored
4. Day 0: starting a new project
5. Phase by phase: deal to release
6. A developer's ticket, step by step
7. The gates: what is checked before a PR
8. Change requests
9. Team rules
10. Skills and commands reference
11. Troubleshooting common denials
12. Glossary
13. Status and what is still pending


# 1. The model in one page

Every client project uses three places, and nothing is copied between them. Agents read Notion and the tracker live through MCP (the connector that lets Claude read and write those tools as the signed-in person).

| Place | Holds | Written by |
|---|---|---|
| **Notion** | What the project is: Opportunity Packet, BRD, PRD, Technical Design, project-wide glossary and decisions, estimates. | Sales, Product Owners, Architect |
| **Linear** (or Jira / Asana) | What to build next: one parent issue per feature, one child issue per repo with work for it. | Project Manager, Architect |
| **Git repos** | The code, plus the proof it was built right: specs, design briefs, prototype approvals, QA reports. | Developers, QA |

**One repo or several.** A small product lives in one repo (role *single*). A larger one is split into `<client>-fe`, `<client>-be` and `<client>-infra`, each self-contained. They share context through Notion and Linear, not through each other. The backend repo owns the **API contract** (`api/openapi.yaml`), which is how the frontend and backend agree without waiting for each other.

**Why specs, approvals and QA reports live in git.** The `rs-guard` plugin checks them before a pull request can open, and reviewers see them in the diff. Those checks run locally and cannot call Notion or Linear, so the evidence has to be in the repo.

Sales: deal & packet → PO: PRD & prototypes → Design: Figma → Architect: tech design, repos, specs → PM: Linear plan → Devs: build per repo → QA & release

# 2. Roles and responsibilities

Names are the Compass example; swap in your own team. Commands that start with `/` are typed by a person; agents never start them on their own.

### Sales: Deep, Chris

- **Owns:** discovery, proposal, pricing input, contract.

- **Does:** researches the prospect, runs discovery calls with the client, turns transcripts and emails into the Opportunity Packet, writes the BRD, pitches, drafts the SOW.
- **Skills:** `rs-research:company-research`, `rs-scoping:opportunity-packet`, `rs-scoping:brd-generator`, `rs-sales:gamma-prototype-proposal-deck`, `rs-scoping:estimation-generator`, `rs-sales:contract-creator`.
- **Stores in:** Notion (Opportunity Packet database, BRD); Gamma (deck); Word (contract); `.xlsx` (estimate).
- **Hands off when:** the contract is signed.

### Product Owners: Lekshmi, Astin, Deep

- **Owns:** the PRD and the clickable prototypes; client sign-off on both.
- **Does:** writes the PRD from the Opportunity Packet and approved BRD; builds clickable prototypes from it; walks the client through them and records the approval (email, Notion comment or meeting note, with a link).
- **Skills:** `rs-scoping:prd-generator`, `rs-prototype:html-prototype-generator` (mobile) or `rs-prototype:nextjs-prototype-generator` (web).
- **Stores in:** Notion (PRD page, approval evidence); prototype links (Artifact, Vercel or HTML file) recorded on the PRD page.
- **Hands off when:** the client approves the PRD and the prototypes.

### Designers

- **Owns:** Figma screens and the design system.
- **Skills:** `rs-design:saas-uiux-designer-figma`, `rs-design:design-reference-to-css-spec`; in the frontend repo, `rs-delivery:design-guidelines`.
- **Stores in:** Figma; the design system goes into the frontend repo as `Design Guidelines/design-system.md`.

### Architect / Lead: Vishnu

- **Owns:** the technical design, the repo split, the API contract, the spec packages, change requests.
- **Does:** writes the technical design; creates the repos from the template and runs the setup script in each; sets GitHub permissions; fixes the API contract; runs `spec-generator` in each repo for its slice and reviews every spec; runs the design interview for change requests.
- **Skills:** `rs-scoping:technical-design-generator`, `rs-delivery:repo-knowledge-base`, `rs-delivery:spec-generator`, `/grill-with-docs`, `/to-spec`.
- **Stores in:** Notion (technical design); each repo (`specs/`, `api/openapi.yaml`, `.rs-harness.json`, `docs/agents/`).

### Project Manager: Fairuz

- **Owns:** the Linear project, sprint pacing, client status updates.
- **Does:** turns the PRD into milestones, parent issues and per-repo child issues; approves the plan before it is created; tracks progress; reports to the client.
- **Skills:** `rs-project-management:prd-to-linear-sprint-mapper` (V3 for multi-repo projects).
- **Stores in:** Linear.

### Developers: Amsu (backend), Nakul (frontend), Dinesh (DevOps / infra)

- **Owns:** their own repo only (`compass-be`, `compass-fe`, `compass-infra`).

- **Does:** one branch per child issue; test-first implementation from `tasks.md`; QA; review; PR. Section 6 has the exact steps.
- **Skills:** `rs-delivery:ticket-intake`, `/implement`, `/to-tickets` + `/implement-spec`, gstack `/ship --no-pr` and `/review`, `rs-delivery:qa-verify-report`, `rs-delivery:ship-with-ticket`; frontend also `rs-delivery:feature-design-brief` and `rs-delivery:feature-prototype`.
- **Stores in:** their repo; PRs on GitHub; status comments on the Linear child issue.
- **Dinesh also:** deploys staging and production.

### QA Lead: Astin

- **Owns:** the test strategy, QA standards per PR, release regression.
- **Does:** writes the test strategy and coverage matrix from the approved PRD; sets what per-PR QA must cover; runs full regression on staging before each release.
- **Skills:** `rs-qa:test-planner`, `rs-qa:test-case-generator`, `rs-qa:test-executor`; per PR, `rs-delivery:qa-verify-report`.
- **Stores in:** Notion (test plan); repos (`Execution Reports/`).

### Client: Steve (Compass)

- Approves the BRD, the PRD and the prototypes; runs UAT on staging; requests changes through the PM.
- Sees only the Notion pages shared with them. Pricing and margin pages are never shared.

# 3. Where everything is stored

## Notion: one project home per client

| Page | Contents | Owner |
|---|---|---|
| Project home | Links to everything below, the repos and the Linear project. | PM |
| Opportunity Packet | Transcripts, emails, notes from discovery (a Notion database). | Sales |
| BRD | Business case, objectives, scope, risks. | Sales |
| PRD (approved) | Features with acceptance criteria; prototype links and client approval evidence. | Product Owners |
| Technical Design | Architecture, diagrams, component inventory. | Architect |
| Glossary (project-wide) | Domain terms every repo uses. | Architect |
| Decisions / ADRs (project-wide) | Hard-to-reverse decisions that affect several repos. | Architect |
| Test strategy | Test plan and coverage matrix. | QA Lead |
| Internal: estimate, pricing | Never shared with the client. | Sales |

## Linear

- One project per client, ticket prefix such as `COM`.
- **Parent issue** per feature (`Feature: Monthly report`, e.g. COM-12): PRD link, acceptance criteria, list of children. No work of its own; it closes when all children are done. `/to-spec` may post the agreed spec on it.
- **Child issues** per repo (COM-13 `[infra] …`, COM-14 `[be] …`, COM-15 `[fe] …`): labelled `repo:fe`, `repo:be` or `repo:infra`, 0.5–2 days each, with only that repo's criteria and the branch name to use.
- Design, content, release QA and UAT issues carry a function label instead of a repo label.

## Each repo

| Path | Contents | Written by |
|---|---|---|
| `.rs-harness.json` | Repo role, ticket prefix, MCP servers planning needs. Read by the gates. | Setup script |
| `docs/agents/project-context.md` | Links to the Notion pages, tracker project, sibling repos, API contract path. | Setup script |
| `docs/agents/issue-tracker.md`, `triage-labels.md`, `domain.md` | Where `/to-spec` and `/to-tickets` publish; labels; how to read domain docs. | Template (fill in once) |
| `AGENTS.md`, `CLAUDE.md` | How delivery works here, house rules, the gates. | Template |
| `manifest.yaml` | Repo map: stack, entry points, conventions. | `repo-knowledge-base` |

| | | |
|---|---|---|
| `Ticket Briefs/<ID>.md` | Normalized ticket, parent spec, links to the relevant Notion pages. | `ticket-intake` |
| `specs/<feature>/` | `requirements.md` (EARS criteria, with source frontmatter), `design.md` (design and agreed test seams), `tasks.md` (RED → GREEN tasks); `specs/README.md` index. | `spec-generator` |
| `Design Guidelines/design-system.md` | Palette, type, spacing, components. Frontend only. | `design-guidelines` |
| `Feature Designs/<feature>.md` | Flow, design, entry points, accessibility, and the `## Prototype Approval` record. Frontend only. | `feature-design-brief`, `feature-prototype` |
| `Prototypes/` | Disposable clickable prototypes (if not an Artifact link). | `feature-prototype` |
| `Execution Reports/` | QA verdicts with evidence (browser or API mode). | `qa-verify-report` |
| `GLOSSARY.md`, `docs/adr/` | Repo-specific terms and decisions only (project-wide ones live in Notion). | `/grill-with-docs` |
| `api/openapi.yaml` | The API contract. Backend repo only. | Backend developer |
| `.rs-receipts/` | Local proof that tests and QA passed on a commit. Gitignored, never committed. | `rs-receipt` |

# 4. Day 0: starting a new project

## Once per person (machine setup)

1. Install Claude Code, `git`, `gh` (signed in with `gh auth login`), Node, and Bun 1.3.3 or later. macOS or Linux; Windows users work in WSL.
2. Install gstack at the pinned commit and Matt Pocock's skills. Open any template repo in Claude Code: the session start prints exactly what is missing and the command to fix it. A human runs installers; agents are blocked from doing so.
3. The first time a gstack skill runs it asks about artifacts sync: answer **off**, so nothing about client work leaves the machine. Optionally run `gstack-config set proactive false` so gstack doesn't suggest its own skills on top of the pipeline.
4. Don't add personal Notion or Linear connectors for client work; `rs-delivery` provides them, and duplicates confuse the agent.

## Once per project

1. **PM:** create the Notion project home with the pages in Section 3, and the Linear project with its ticket prefix.
2. **Architect:** decide one repo or several. Create each repo from the RightShift template repo on GitHub (**Use this template**), named `<client>` or `<client>-fe`, `-be`, `-infra`.
3. **Architect, in each repo:** run the setup script. It asks for the role, prefix, tracker and Notion links:

   ```
   node scripts/rs-setup.mjs
   # or non-interactive:
   node scripts/rs-setup.mjs --role be --prefix COM --tracker linear --client compass \
     --notion-home https://www.notion.so/... --prd https://www.notion.so/... \
     --tech-design https://www.notion.so/... --glossary https://www.notion.so/...
   ```

   It writes `.rs-harness.json` and `docs/agents/project-context.md`, then prints the remaining steps.
4. **Architect:** fill in `docs/agents/issue-tracker.md`, commit `.rs-harness.json` and `docs/agents/`.
5. **Architect, on GitHub, each repo:** developers get write access only to their own repo (read-only on the others); protect `main`; add a `CODEOWNERS` file so a repo owner approves every merge.
6. **Backend repo:** create `api/openapi.yaml`, even if nearly empty.
7. **Frontend repo:** run `rs-delivery:design-guidelines` once the design system exists.
8. **Every repo:** run `rs-delivery:repo-knowledge-base` once code exists, to write `manifest.yaml`.

## Once per person, per project

1. Open the repo in Claude Code and trust the folder. Install the plugins:

   ```
   /plugin install rs-delivery@rightshift
   /plugin install rs-guard@rightshift
   /plugin install rs-scoping@rightshift
   /plugin install rs-qa@rightshift
   /plugin install rs-common@rightshift
   /plugin install rs-project-management@rightshift   # PM and Architect
   ```

2. Run `/mcp` and sign in to the Notion and tracker servers that `rs-delivery` provides, with your own account. Your Notion permissions decide what the agent can read. Until you sign in, planning skills are blocked and the session opens with a STOP note.

## Example `.rs-harness.json` files

| Repo | File contents |
|---|---|
| Single repo | `{ "ticket_prefixes": ["COM"] }` (plus `required_mcp` if the project uses Notion) |
| `compass-fe` | `{ "role": "fe", "ticket_prefixes": ["COM"], "required_mcp": ["plugin:rs-delivery:notion", "plugin:rs-delivery:linear-server"] }` |
| `compass-be` | Same with `"role": "be"` |
| `compass-infra` | Same with `"role": "infra"` |

## 5. Phase by phase: deal to release

| # | Phase | Who | Steps and skills | Output and location | Done when |
|---|---|---|---|---|---|
| 1 | Win the deal | Deep, Chris | `company-research` → discovery calls → `opportunity-packet` → `brd-generator` → `gamma-prototype-proposal-deck` → `estimation-generator` (needs a first-cut PRD) → `contract-creator` | Packet and BRD in Notion; deck; estimate; SOW | Contract signed |
| 2 | Define the product | Lekshmi, Astin, Deep | `prd-generator` → client approves → `html-prototype-generator` / `nextjs-prototype-generator` → client clicks through and approves (record the evidence link). Astin starts `test-planner`. | PRD, prototype links, approval evidence, test strategy in Notion | Steve approves PRD and prototypes |
| 3 | Design | Designers | `saas-uiux-designer-figma` → `design-reference-to-css-spec` | Figma; design system in `compass-fe/Design Guidelines/` | Design approved |
| 4 | Architecture and specs | Vishnu | `technical-design-generator` → create repos, setup script, permissions (Section 4) → API contract → `spec-generator` in each repo for its slice → review specs | Tech design in Notion; `specs/` and `api/openapi.yaml` in repos | Specs merged in each repo |
| 5 | Plan the work | Fairuz | `prd-to-linear-sprint-mapper` (V3): milestones, parent issues, per-repo children, API-contract child first, cycles. Approve the plan, then create it. | Linear project | Sprint planned, children assigned |
| 6 | Build | Dinesh, Amsu, Nakul | One branch per child issue, each in their own repo (Section 6). Merge order within a feature: infra → backend → frontend. | PRs, merged code, `Execution Reports/` | All children merged, parent closed |
| 7 | QA and release | Astin, Dinesh, Fairuz | `test-case-generator` → `test-executor` on staging (full regression) → Steve's UAT → Dinesh deploys → Fairuz reports | Regression report; release | Released |

> **Single repo:** phases are the same, but there is one spec package, one design brief per feature, and browser QA covers the whole feature.

## 6. A developer's ticket, step by step

Example: Amsu picks up COM-14, `[be] Report API`, child of COM-12 *Monthly report*. Open Claude Code in the repo (`compass-be`), on a fresh branch.

1. **Branch:** `git switch -c feat/COM-14-report-api` (the issue names the branch). The ticket ID in the branch name is what the gates key on.
2. **Pull the ticket:** ask for `rs-delivery:ticket-intake` (e.g. "start on COM-14"). It writes `Ticket Briefs/COM-14.md`, including the parent's spec and links to the PRD in Notion.
3. **Spec, if your slice doesn't have one yet:** ask for `rs-delivery:spec-generator`. It writes only this repo's part, with frontmatter like:

   ```
   ---
   feature: monthly-report
   tickets: [COM-12, COM-14]
   source: https://www.notion.so/.../Compass-PRD...
   source_edited: 2026-10-10T14:02Z
   repo_role: be
   ---
   ```

   Review it and commit it. Backend tasks include updating `api/openapi.yaml` and a contract test.
4. **Frontend only, before code:** `rs-delivery:feature-design-brief`, then `rs-delivery:feature-prototype`. If the client already approved the PO's prototype for this feature, the prototype skill records that approval (with the evidence link) instead of building a new one; screens it didn't cover still need a quick prototype and approval.
5. **Implement, test-first:** type `/implement specs/monthly-report/` for a small slice, or `/to-tickets` then `/implement-spec` to build several slices in parallel. Every task is one failing test at an agreed seam, then only the code that makes it pass. Code review runs at the end.
6. **Prepare the branch:** type `/ship --no-pr` (gstack). It merges the base branch, runs the tests, reviews and fixes, and pushes. It opens no PR. If it offers to create `TODOS.md`, answer **Skip**.
7. **QA:** ask for `rs-delivery:qa-verify-report` against staging. Backend runs in API mode (calls from the contract), frontend in browser mode, infra skips QA. Approve the report, commit it, then record the receipt:

   ```
   rs-receipt qa --report "Execution Reports/<file>.md" --p0-failed 0
   ```

   (The full `node …/rs-receipt.cjs` path is printed at session start.)
8. **Final review:** type `/review` (gstack). It must finish clean on the exact final code. If it changes code, redo QA and the receipt.
9. **Tests receipt:** `rs-receipt verify -- <your test command>`, e.g. `rs-receipt verify -- npm test`.
10. **Open the PR:** ask for `rs-delivery:ship-with-ticket`. It checks `rs-receipt status`, writes the PR body, shows it to you for approval, opens the PR (closes COM-14) and comments on the Linear issue.

> **Order matters.** `/review` and the receipts are tied to the exact code. Any change after them, even adding a report file, makes them stale and the PR is refused until they are redone. A draft PR (`gh pr create --draft`) can be opened at any time to share work early.

**Chores** (no ticket ID in the branch, e.g. `chore/bump-deps`): only tests and `/review` are required.

# 7. The gates: what is checked before a PR

`rs-guard` checks these when anyone, human-asked or agent, runs `gh pr create` (non-draft) or `gh pr ready`. A failed check denies the command and names the missing step.

| Check | single | fe | be | infra |
|---|---|---|---|---|
| Tests passed on this exact commit, clean tree (verify receipt) | yes | yes | yes | yes |
| gstack `/review` clean on the current code | yes | yes | yes | yes |
| Prototype approved in `Feature Designs/` (ticket branches) | yes | yes | no | no |
| QA receipt with no failing P0 (ticket branches) | yes | browser | API | no |
| A spec lists the ticket and cites its source (ticket branches) | no | yes | yes | yes |
| Required tools installed (gstack, Matt's skills, Bun, git, gh) | yes | yes | yes | yes |

### Always blocked for agents

- Force-push, push to `main` / the default branch, `git reset --hard`, `git clean -f`, `git branch -D`, `git checkout .`, `git restore .`. A human runs them if really needed.
- Running the tool installer (`rs-doctor --install`).
- A `cd` to a path only known at run time before `gh pr create` or `git push`, and `gh pr … --repo` naming another repository. The gates check the repo a command actually acts on.

### Planning gate (MCP sign-in)

While a server listed in `required_mcp` is signed out, these are blocked, whether an agent starts them or a person types them: `kickoff`, `ticket-intake`, `spec-generator`, `feature-design-brief`, `/grill-with-docs`, `/to-spec`, `/to-tickets`. Coding, tests and PRs keep working offline.

Check what is recorded for the current commit any time with `rs-receipt status`: it shows the repo role, what its ticket branches need, both receipts and the `/review` verdict.

# 8. Change requests

A request from the client that is not in the approved PRD:

1. The client raises it with Fairuz, who creates a new parent issue.
2. Vishnu runs the design interview: type `/grill-with-docs` (reads the ticket and Notion, asks until the design is settled, including any API change).
3. Type `/to-spec`: the agreed spec is posted on the parent issue.
4. Fairuz (sprint mapper) or Vishnu (`/to-tickets`) creates the per-repo children.
5. Each repo then follows Section 6. Only change requests need a new prototype.

## 9. Team rules

- **Stay in your repo.** A frontend task that needs a backend change gets its own `repo:be` child issue. It is never slipped into the frontend PR.
- **Contract first.** Any API change starts as an edit to `api/openapi.yaml` through a backend child issue; the frontend mocks the API from that file.
- **Merge order.** Infra, then backend (additive or behind a feature flag), then frontend.
- **Test-first, always.** One failing test at an agreed seam, then the minimal code. Never test at a seam that wasn't agreed in `design.md`.
- **Diagnose before fixing.** Use `mattpocock-skills:diagnosing-bugs`: a reproducible failing signal first, then the fix with a regression test.
- **No invented scope.** Anything not in the approved source is tagged (`PRD GAP:`, `DECISION NEEDED:`) and raised, never guessed.
- **Human gates are real gates.** No implicit approvals, no routing around a denial.
- **Link, don't copy.** Specs and briefs cite Notion pages; they don't paste them.
- **Commits** follow Conventional Commits (`feat:`, `fix:`, `docs:`…).
- **Client data** stays in shared-with-care Notion pages; never paste credentials or personal data into reports.

## 10. Skills and commands reference

| Skill or command | What it does | Used by |
|---|---|---|
| **rs-research, rs-sales, rs-scoping** | | |
| `rs-research:company-research` | Prospect package: company overview, news, value ideas. | Sales |
| `rs-scoping:opportunity-packet` | Notion database from transcripts and notes. | Sales |
| `rs-scoping:brd-generator` | Client-facing BRD. | Sales |
| `rs-sales:gamma-prototype-proposal-deck` | Proposal deck in Gamma. | Sales |
| `rs-scoping:estimation-generator` | Fills the estimation sheet from a PRD. | Sales, Architect |
| `rs-sales:contract-creator` | First-draft SOW / MSA / NDA. | Sales |
| `rs-scoping:prd-generator` | Dual-layer PRD from the packet (and BRD). | Product Owners |
| `rs-scoping:technical-design-generator` | Architecture, diagrams, component inventory. | Architect |
| **rs-prototype, rs-design** | | |
| `rs-prototype:html-prototype-generator` | Clickable mobile prototype (single HTML file). | Product Owners |
| `rs-prototype:nextjs-prototype-generator` | Deployable Next.js demo app. | Product Owners |
| `rs-design:saas-uiux-designer-figma` | Approval-gated UI design build in Figma. | Designers |
| `rs-design:design-reference-to-css-spec` | CSS design-system spec from a reference. | Designers |
| **rs-project-management** | | |
| `prd-to-linear-sprint-mapper` | Linear plan: milestones, parent + per-repo children (V3), cycles. | PM |
| **rs-delivery** | | |
| `kickoff` | Entry point: reads project context, picks New application or Single feature, names the first step. | Anyone |
| `repo-knowledge-base` | Indexes the repo into `manifest.yaml`. | Architect |
| `design-guidelines` | Writes the repo's design system. | Frontend, Designers |
| `ticket-intake` | Ticket (and parent) into a Ticket Brief with Notion links. | Developers |

| | | |
|---|---|---|
| `spec-generator` | Requirements / design / tasks per feature; per-repo slice; source frontmatter. | Architect, Developers |
| `feature-design-brief` | Flow, design, entry points, accessibility. | Frontend |
| `feature-prototype` | Clickable prototype and the approval record (or reuses an earlier approval). | Frontend |
| `qa-verify-report` | Live QA against the criteria; browser or API mode. | Developers, QA |
| `ship-with-ticket` | Opens the PR linked to the ticket, comments on the ticket. | Developers |
| **rs-qa** | | |
| `rs-qa:test-planner` → `test-case-generator` → `test-executor` | Test strategy, test cases, execution on staging. | QA Lead |
| **Matt Pocock's skills (typed by a person)** | | |
| `/grill-with-docs` | Design interview; updates glossary and ADRs. | Architect |
| `/to-spec` | Posts the agreed spec to the tracker. | Architect |
| `/to-tickets` | Splits a spec into tracker tickets. | Architect, Developers |
| `/implement`, `/implement-spec` | Test-first build of one slice, or several from tickets; code review at the end. | Developers |
| **gstack (typed by a person)** | | |
| `/ship --no-pr` | Merge base, test, review and fix, push. No PR. | Developers |
| `/review` | Final pre-PR review; its clean verdict is a gate. | Developers |
| **rs-guard** | | |
| `rs-receipt verify -- <cmd>` | Runs the tests and records a receipt for this commit. | Developers |
| `rs-receipt qa --report … --p0-failed <n>` | Records a committed QA report's verdict. | Developers |
| `rs-receipt status` | Shows role, receipts, review verdict for this commit. | Anyone |

## 11. Troubleshooting common denials

| Message starts with | Do this |
|---|---|
| `[rs-guard:verify-receipt] … no verify receipt` | Commit your work, then `rs-receipt verify -- <tests>`. |
| `… uncommitted changes to tracked files` | Commit or stash; receipts must match committed code. |
| `[rs-guard:review-gate] … STALE` or `no gstack /review` | Type `/review` again on the final code; it must finish clean. |

| | |
|---|---|
| `[rs-guard:prototype-gate] …` | Frontend: run `feature-design-brief` and `feature-prototype`; make sure the brief's `ticket:` / `tickets:` lists your ticket. |
| `[rs-guard:qa-receipt] …` | Run `qa-verify-report`, commit the report, record `rs-receipt qa`. |
| `[rs-guard:spec-source] … no specs/*/requirements.md lists …` | Run `spec-generator` for this repo, or add your ticket ID to the spec's `tickets:` if you split a ticket. |
| `[rs-guard:mcp] Blocked: … reads project context through MCP` | Run `/mcp` and sign in to the named server. |
| `[rs-guard:dependencies] … required tools … are missing` | Install what the message lists (a human runs the commands). |
| `… cannot tell which repository …` | Run the command from inside the target repo, without `cd` to a variable path or `--repo`. |
| `… .rs-harness.json has "role": …` | Use `fe`, `be` or `infra`, or remove the role for a single repo. |

## 12. Glossary

| Term | Meaning |
|---|---|
| Parent / child issue | A feature's tracking ticket, and the per-repo tickets that build it. |
| Role | A repo's part in a multi-repo project (`fe`, `be`, `infra`); decides which gates apply. |
| Slice | The part of a feature one repo builds; what `spec-generator` writes in that repo. |
| API contract | `api/openapi.yaml` in the backend repo; the agreement both sides build against. |
| Test seam | The agreed public boundary where a feature's tests run (UI, API or plan output). |
| EARS | The requirement sentence pattern (`WHEN … THE SYSTEM SHALL …`) used in `requirements.md`. |
| Receipt | A local file proving tests or QA passed on an exact commit; any new commit needs a new one. |
| Gate | A check `rs-guard` runs before a PR or a risky command. |
| MCP | The connector that lets Claude read Notion and the tracker as the signed-in person. |
| Source frontmatter | The block at the top of `requirements.md` naming the approved document and tickets it came from. |

## 13. Status and what is still pending

> **Released (v4.0.0, available now):** the single-repo harness: the gates, gstack `/ship` and `/review`, Matt Pocock's skills, test-first specs.
>
> **Built but not yet released (branch `feat/multi-repo`, planned as v5):** repo roles, the MCP planning gate, Notion project context, `spec-generator` v4 slices and the spec-source gate, API-mode QA, the sprint mapper's per-repo tickets, reusing an earlier prototype approval, and the setup script. Until v5 ships, test projects point their marketplace `ref` at `feat/multi-repo`.
>
> **Before v5:** the manual validation run on three throwaway repos (checklist in the *Multi-Repo Delivery Framework* doc). Open decisions: whose Notion workspace holds client pages; whether the client gets read access to the PRD; confirming the flow for the first Jira or Asana client.